

**МИНИСТЕРСТВО НАУКИ, ВЫСШЕГО ОБРАЗОВАНИЯ И ИННОВАЦИЙ
КЫРГЫЗСКОЙ РЕСПУБЛИКИ
КЫРГЫЗСКО-ГЕРМАНСКИЙ ИНСТИТУТ ПРИКЛАДНОЙ ИНФОРМАТИКИ**

ОТЧЕТ

о преддипломной практике
место прохождения практики: ИП «Тенизбаев Улукбек Кубанович»

Выполнил:

студент группы WIN-1-22

Бактыбеков Нурсултан Бактыбекович

Руководитель практики:

к.т.н. доцент

Самсалиев Анвар Амантаевич _____

Бишкек 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
ОСНОВНАЯ ЧАСТЬ	4
1. Организационные вопросы оформления на предприятии	4
2. Знакомство со структурой и характером деятельности профильной организации	5
3. Получение индивидуального задания на практику, его детализация	6
4. Выполнение индивидуального задания	7
5. Выполнение задания по теме ВКР, его детализация	8
6. Анализ полученных результатов	10
7. Оформление отчета по практике	11
ВЫВОДЫ	11
ПРИЛОЖЕНИЯ	13
Приложение 1. Структура проекта и архитектура	13
Приложение 2. Исходный код смарт-контракта (lib.rs)	13
Приложение 3. Описание процесса деплоя и запуска приложения	17

ВВЕДЕНИЕ

Преддипломная практика является важнейшей, неотъемлемой и обязательной частью учебного процесса, завершающей этап теоретического обучения в высшем учебном заведении. Основная цель преддипломной практики заключается в закреплении, систематизации и углублении теоретических знаний, полученных студентом за все годы обучения в институте. Помимо этого, практика направлена на приобретение и оттачивание профессиональных навыков самостоятельной работы в реальных производственных условиях, что позволяет студенту адаптироваться к будущей профессиональной деятельности и понять специфику работы в IT-сфере.

Преддипломная практика выступает в роли фундаментального подготовительного этапа к выполнению выпускной квалификационной работы (ВКР). Исходя из этого, главной и приоритетной задачей студента в этот период является сбор, всестороннее изучение, анализ и систематизация данных, непосредственно связанных с выбранной темой ВКР. Это включает в себя изучение предметной области, анализ существующих решений на рынке, выявление их достоинств и недостатков, а также обоснование выбора конкретного стека технологий для реализации собственного проекта.

Тема моей выпускной квалификационной работы звучит следующим образом: «Разработка системы электронного голосования на базе технологии блокчейн». Актуальность выбранной темы в современных реалиях не вызывает сомнений. В эпоху цифровизации и перехода многих процессов в онлайн-пространство, вопросы обеспечения прозрачности, неизменности, достоверности и абсолютной безопасности процесса голосования становятся как никогда острыми. Традиционные электронные системы голосования, основанные на централизованных базах данных, подвержены различным рискам: от хакерских атак и несанкционированного доступа до намеренной фальсификации результатов со стороны администраторов системы. Решение этих проблем может быть эффективно достигнуто с использованием децентрализованных технологий, распределенных реестров и смарт-контрактов, которые гарантируют, что каждый отданный голос будет учтен корректно и его невозможно будет изменить или удалить постфактум.

Для успешного прохождения практики и достижения поставленных целей были сформулированы следующие ключевые задачи:

- Изучение правил внутреннего трудового распорядка, техники безопасности, охраны труда и пожарной безопасности на предприятии ИП «Тенизбаев Улукбек Кубанович».
- Подробное ознакомление со структурой предприятия, спецификой его деятельности, текущими проектами и корпоративной культурой.
- Сбор, систематизация и анализ материалов, необходимых для проектирования и последующей разработки децентрализованного приложения (dApp) для электронного голосования.

- Глубокое изучение и практическое освоение стека технологий, выбранного для реализации ВКР, а именно: блокчейн-платформы Solana, фреймворка для разработки смарт-контрактов Anchor, языка программирования Rust, а также современных инструментов для создания пользовательских интерфейсов — библиотеки React и сборщика Vite.
- Непосредственная разработка программного продукта, включающая написание смарт-контрактов, создание клиентской части, интеграцию с криптовалютными кошельками и реализацию сервиса индексации данных.
- Проведение комплексного тестирования разработанного программного продукта, выявление и устранение ошибок, а также анализ полученных результатов на предмет соответствия первоначальным требованиям технического задания.

В качестве основных источников информации в ходе прохождения практики использовалась официальная техническая документация по блокчейну Solana и фреймворку Anchor, научные статьи, публикации в специализированных IT-изданиях, посвященные применению смарт-контрактов в системах электронного голосования, а также материалы, касающиеся основ криптографии и информационной безопасности.

ОСНОВНАЯ ЧАСТЬ

1. Организационные вопросы оформления на предприятии

Прохождение преддипломной практики началось с решения организационных вопросов оформления на базе предприятия ИП «Тенизбаев Улукбек Кубанович». В первый день практики, в соответствии с установленным регламентом, мною был пройден вводный инструктаж по технике безопасности, охране труда и пожарной безопасности. Инструктаж проводился ответственным специалистом компании и включал в себя подробное разъяснение правил поведения на рабочем месте, мер по предотвращению несчастных случаев и порядка действий в экстренных ситуациях.

Я внимательно изучил инструкции по охране труда, схемы аварийных проходов и выходов, а также расположение пожарного инвентаря. Особое внимание было уделено правилам внутреннего трудового распорядка организации: режиму рабочего времени, времени отдыха, субординации и корпоративным стандартам общения.

После успешного прохождения всех инструктажей, мне было выделено индивидуальное рабочее место в офисе компании. Рабочее место было полностью укомплектовано современной вычислительной техникой, необходимой для выполнения поставленных задач. Был проведен дополнительный инструктаж по правилам и нормам охраны труда при работе с персональным компьютером, включающий рекомендации по правильной организации рабочего пространства, оптимальному освещению и

необходимости делать регулярные перерывы для снижения нагрузки на зрение и опорно-двигательный аппарат.

2. Знакомство со структурой и характером деятельности профильной организации

Следующим этапом стало детальное знакомство со структурой и характером деятельности ИП «Тенизбаев Улукбек Кубанович». Данная организация позиционирует себя как современная и динамично развивающаяся компания, специализирующаяся на разработке высококачественного программного обеспечения, оказании широкого спектра IT-услуг и профессиональном консалтинге в сфере информационных технологий.

В ходе ознакомления я изучил основные направления деятельности компании. К ним относятся: разработка веб-приложений и корпоративных порталов, создание мобильных приложений для платформ iOS и Android, проектирование и внедрение систем автоматизации бизнес-процессов, а также исследование и разработка решений на базе передовых технологий, таких как блокчейн, искусственный интеллект и машинное обучение.

Я ознакомился со структурой организации, которая включает в себя отдел разработки (frontend, backend, mobile), отдел тестирования и контроля качества (QA), отдел дизайна (UI/UX), а также отдел проектного менеджмента и бизнес-аналитики. Было интересно наблюдать за тем, как организовано взаимодействие между различными отделами в процессе работы над реальными коммерческими проектами. Компания активно применяет гибкие методологии разработки программного обеспечения, в частности Scrum и Kanban, что позволяет эффективно управлять рабочими процессами, оперативно реагировать на изменения требований заказчиков и поставлять качественный продукт в установленные сроки.

Также я детально ознакомился с техническими средствами, предоставленными на моем выделенном рабочем месте. Для успешного выполнения задач, связанных с разработкой блокчейн-приложения, потребовалась настройка специфического рабочего окружения. Мною были установлены и настроены следующие программные пакеты и инструменты: операционная система на базе ядра Linux (для обеспечения максимальной совместимости с инструментами разработки), среда выполнения Node.js, язык программирования Rust (являющийся основным языком для написания смарт-контрактов в сети Solana), интерфейс командной строки Solana CLI, а также фреймворк Anchor, значительно упрощающий процесс разработки и тестирования смарт-контрактов. В качестве интегрированной среды разработки (IDE) был выбран Visual Studio Code с набором необходимых расширений для работы с Rust и React.

3. Получение индивидуального задания на практику, его детализация

Важнейшим этапом первой недели практики стало получение индивидуального задания. Совместно с руководителем практики от профильной организации мы провели установочную встречу, на которой подробно обсудили тему моей выпускной квалификационной работы и определили объем работ, который необходимо выполнить в рамках преддипломной практики. По итогам встречи было составлено и официально утверждено техническое задание на разработку прототипа децентрализованной системы электронного голосования на базе блокчейна Solana.

В процессе работы над техническим заданием была проведена детальная декомпозиция поставленной задачи. Были четко определены и задокументированы функциональные требования к разрабатываемой системе. К основным функциональным возможностям были отнесены:

- Создание новых сессий голосования (Proposals) пользователями с указанием уникального идентификатора и текстового описания (сути выносимого на голосование вопроса).
- Регистрация и аутентификация пользователей в системе посредством подключения криптовалютных кошельков, поддерживающих сеть Solana (например, Phantom, Solflare).
- Обеспечение возможности для зарегистрированных пользователей отдавать свой голос («За» или «Против») по активным голосованиям.
- Реализация надежного механизма защиты от двойного голосования, гарантирующего, что один уникальный кошелек может проголосовать в рамках одного конкретного голосования только один раз.
- Автоматический, прозрачный и децентрализованный подсчет голосов с отображением актуальных результатов в режиме реального времени.
- Возможность закрытия сессии голосования её создателем.

Помимо функциональных требований, были сформулированы и нефункциональные требования, определяющие качественные характеристики системы. К ним относятся:

- Высокая производительность и минимальные задержки при обработке транзакций, что достигается за счет использования высокоскоростного блокчейна Solana.
- Обеспечение безопасности и неизменности данных: все голоса и результаты должны надежно храниться в блокчейне без возможности их фальсификации.
- Интуитивно понятный, современный и отзывчивый пользовательский интерфейс веб-приложения, обеспечивающий комфортную работу пользователей независимо от уровня их технической подготовки.
- Масштабируемость архитектуры, позволяющая в будущем расширять функционал системы и выдерживать рост числа пользователей без существенного снижения производительности.

4. Выполнение индивидуального задания

Приступив к выполнению индивидуального задания, я начал с проведения глубокого исследования предметной области. Были всесторонне проанализированы существующие решения в сфере электронного голосования, как традиционные, так и основанные на различных блокчейн-платформах. Анализ показал, что традиционные централизованные системы электронного голосования обладают рядом критических уязвимостей, главной из которых является необходимость полного доверия к центральному органу, администрирующему систему. Это создает риски внутреннего мошенничества, манипуляций с базой данных и делает систему привлекательной мишенью для кибератак.

В качестве альтернативы были рассмотрены децентрализованные системы голосования. Технология блокчейн, благодаря своей распределенной природе, криптографической защите и использованию механизмов консенсуса, способна решить проблему доверия, обеспечив прозрачность и невозможность изменения однажды записанных данных. При выборе конкретной блокчейн-платформы для реализации проекта проводилось сравнение популярных сетей, таких как Ethereum и Solana. Выбор пал на Solana по нескольким веским причинам. Во-первых, Solana обладает уникальным механизмом консенсуса Proof-of-History (PoH), который в сочетании с Proof-of-Stake (PoS) обеспечивает невероятно высокую пропускную способность (до 65 000 транзакций в секунду) и минимальные задержки подтверждения транзакций (около 400 мс). Во-вторых, стоимость транзакций в сети Solana несоизмеримо ниже, чем в Ethereum, что делает экономически целесообразным проведение массовых голосований с участием тысяч или миллионов избирателей.

На основе проведенного анализа была спроектирована комплексная архитектура системы электронного голосования. Архитектура состоит из трех основных логических компонентов:

1. **Блокчейн-часть (Smart Contracts / Programs):** Набор программ (смарт-контрактов), развернутых в сети Solana, которые инкапсулируют всю бизнес-логику системы голосования. Они отвечают за создание голосований, прием голосов, валидацию транзакций, контроль уникальности голосов и хранение итоговых результатов.
2. **Клиентская часть (Frontend):** Одностраничное веб-приложение (SPA), с которым непосредственно взаимодействуют пользователи. Оно отвечает за отображение интерфейса, взаимодействие с криптовалютными кошельками пользователей (подписание транзакций) и отправку запросов к смарт-контрактам в блокчейне через RPC-узлы.
3. **Серверная часть / Индексатор (Block-scanner):** Опциональный, но важный компонент для оптимизации работы клиентского приложения. Это фоновый сервис, который непрерывно сканирует блокчейн Solana на предмет транзакций, связанных с нашими смарт-контрактами, и кэширует

актуальное состояние голосований в традиционную реляционную или NoSQL базу данных. Это позволяет клиентскому приложению быстро получать списки всех голосований и их результаты без необходимости выполнения ресурсоемких запросов к RPC-узлам блокчейна.

5. Выполнение задания по теме ВКР, его детализация

Большая часть времени, отведенного на преддипломную практику, была посвящена самому трудоемкому и важному этапу — практической разработке программного продукта в строгом соответствии с составленным техническим заданием и спроектированной архитектурой. Этот процесс был разбит на несколько логических подэтапов.

Разработка смарт-контрактов (блокчейн-часть): Для написания смарт-контрактов под блокчейн Solana использовался язык программирования Rust и фреймворк Anchor. Anchor — это мощный инструмент, который предоставляет удобные абстракции над низкоуровневым API Solana, автоматизирует процесс сериализации и десериализации данных (с использованием формата Borsh), а также берет на себя рутинные задачи по валидации аккаунтов и проверке прав доступа, что значительно ускоряет разработку и снижает вероятность допущения критических уязвимостей.

В рамках проекта был разработан смарт-контракт `e_voting`. Основной сущностью контракта является структура данных `Proposal`, которая представляет собой отдельное голосование. Для хранения данных `Proposal` в блокчейне использовалась концепция Program Derived Addresses (PDA) — специальных адресов, генерируемых на основе семян (seeds) и идентификатора программы. Это позволяет детерминированно вычислять адреса аккаунтов для каждого голосования без необходимости хранения дополнительных таблиц маршрутизации.

Был реализован следующий функционал смарт-контракта:

- Инstrukция `create`: позволяет пользователю инициализировать новое голосование. На вход принимается уникальный идентификатор (`pid`) и текстовое описание (`description`). Внутри инструкции происходит инициализация нового аккаунта PDA для хранения состояния голосования. Проверяется ограничение на максимальную длину описания. Аккаунт `Proposal` сохраняет информацию о создателе, описании, статусе голосования (активно/закрыто) и начальных счетчиках голосов «За» и «Против», равных нулю.
- Инstrukция `vote_yes` и `vote_no`: позволяют избирателям отдать свой голос. Для предотвращения двойного голосования была реализована отдельная структура данных `Vote`, которая также хранится как PDA-аккаунт. Адрес этого аккаунта вычисляется на основе публичного ключа кошелька избирателя и идентификатора голосования. При попытке проголосовать смарт-контракт проверяет, существует ли уже аккаунт `Vote` для данной

пары «избиратель-голосование». Если да, транзакция отклоняется. Если нет, создается новый аккаунт `Vote`, фиксирующий факт голосования, а соответствующие счетчики (`yes_votes` или `no_votes`) в аккаунте `Proposal` инкрементируются. Также была добавлена проверка статуса голосования — голос принимается только в том случае, если голосование активно.

Разработка клиентской части (Frontend): Для создания современного и отзывчивого пользовательского интерфейса применялась библиотека `React` в связке с быстрым сборщиком `Vite`. Использование `TypeScript` позволило повысить надежность кода за счет строгой статической типизации и избежать множества потенциальных ошибок на этапе разработки.

Одной из главных задач при разработке фронтенда была интеграция с блокчейном `Solana`. Для этого использовались библиотеки `@solana/web3.js` и `@project-serum/anchor` (клиентская часть фреймворка `Anchor`). Был реализован механизм подключения криптовалютных кошельков пользователей. Приложение поддерживает различные популярные кошельки, такие как `Phantom`, с помощью стандарта `Solana Wallet Adapter`.

Пользовательский интерфейс разделен на несколько логических экранов:

- Главная страница: отображает список всех доступных голосований, отсортированных по дате создания или активности.
- Страница создания голосования: содержит форму для ввода описания нового голосования. При отправке формы приложение формирует транзакцию, вызывает инструкцию `create` смарт-контракта и запрашивает у пользователя подпись транзакции через его кошелек.
- Страница голосования: отображает подробную информацию о выбранном голосовании, текущие результаты и кнопки для голосования «За» или «Против». При нажатии на кнопку приложение аналогичным образом формирует транзакцию вызова инструкции `vote_yes` или `vote_no`. После успешного подтверждения транзакции в блокчейне интерфейс автоматически обновляет отображаемые результаты.

Разработка сервиса блок-сканера: Для повышения производительности клиентского приложения был разработан небольшой фоновый сервис — блок-сканер, написанный на `Node.js`. Извлечение списка всех голосований напрямую из блокчейна `Solana` может быть медленным процессом, особенно при большом количестве записей, так как требует получения и десериализации данных всех PDA-аккаунтов программы.

Блок-сканер решает эту проблему путем подписки на события сети через вебсокеты (`WebSockets`). Он отслеживает все новые транзакции, в которых участвует программа `e_voting`. При обнаружении релевантной транзакции (создание голосования или отдача голоса), сканер парсит её содержимое, извлекает обновленные данные и сохраняет их в локальную базу данных (например, `MongoDB` или `PostgreSQL`). Клиентское приложение, в свою

очередь, запрашивает список голосований у API блок-сканера, получая данные практически мгновенно.

Тестирование программного продукта: Завершающим этапом разработки стало тщательное тестирование созданной системы. Тестирование проводилось на нескольких уровнях:

- Модульное тестирование смарт-контрактов: были написаны тесты на языке TypeScript с использованием фреймворка Mocha и библиотеки Chai (стандартный подход при разработке с Anchor). Тесты разворачивали локальный кластер Solana (solana-test-validator), деплоили смарт-контракт и эмулировали различные сценарии взаимодействия. Были протестированы как успешные пути (создание голосования, успешное голосование), так и краевые случаи и обработка ошибок (попытка двойного голосования, голосование в закрытой сессии, превышение лимитов строк). Все тесты успешно пройдены, что подтверждает корректность бизнес-логики контракта.
- Интеграционное тестирование: проверялось корректное взаимодействие всех компонентов системы — фронтенда, блок-сканера и локального блокчейна. Были симулированы действия реальных пользователей в браузере.
- Развертывание в тестовой сети (Devnet): после успешного локального тестирования смарт-контракт был развернут в публичной тестовой сети Solana Devnet. Это позволило проверить работу приложения в условиях, максимально приближенных к реальной эксплуатации, протестировать скорость обработки транзакций реальными валидаторами и оценить стабильность работы интеграции с кошельком Phantom в сетевом окружении.

Также в рамках этого этапа было составлено краткое, но емкое руководство пользователя, описывающее процесс установки расширения кошелька, получение тестовых токенов SOL в Devnet и пошаговые инструкции по использованию основных функций приложения.

6. Анализ полученных результатов

Анализ результатов, полученных в ходе выполнения индивидуального задания, позволяет сделать вывод об успешном достижении поставленных целей. Разработанный прототип децентрализованной системы электронного голосования полностью соответствует требованиям, заявленным в техническом задании.

Применение технологии блокчейн, и в частности платформы Solana, позволило элегантно и эффективно решить фундаментальные проблемы электронного голосования. Механизмы консенсуса и криптографическая защита обеспечили высочайший уровень безопасности: каждый голос записывается в неизменяемый реестр, что делает любые попытки фальсификации или подтасовки результатов математически невозможными.

Прозрачность системы абсолютна — любой желающий может просмотреть исходный код смарт-контракта и самостоятельно верифицировать каждую транзакцию в обозревателе блоков (Solana Explorer).

Тестирование показало высокую производительность и отказоустойчивость системы. Использование фреймворка Anchor позволило создать безопасный и оптимизированный код смарт-контракта, надежно защищенный от распространенных уязвимостей, таких как атаки повторного входа или переполнение буфера. Клиентское приложение демонстрирует быструю и плавную работу, а интеграция с блок-сканером обеспечивает мгновенную загрузку данных без перегрузки RPC-узлов сети.

Полученные практические результаты, разработанный исходный код и накопленный опыт станут прочным фундаментом для написания практической главы моей выпускной квалификационной работы.

7. Оформление отчета по практике

На заключительном этапе преддипломной практики я занимался систематизацией всех собранных материалов, черновиков и набросков. В соответствии с методическими требованиями и стандартами оформления документации КГИПИ, был написан и структурирован настоящий отчет о прохождении практики. Текст отчета был вычитан и отформатирован в соответствии с заданным шаблоном (Typst). Также были подготовлены материалы для предстоящей публичной защиты результатов практики перед комиссией.

ВЫВОДЫ

Прохождение преддипломной практики в ИП «Тенизбаев Улукбек Кубанович» стало для меня исключительно полезным и ценным опытом. За время практики мне удалось не только закрепить и расширить теоретические знания, полученные в стенах института, но и применить их на практике при решении сложной и нетривиальной инженерной задачи.

В ходе выполнения индивидуального задания я глубоко погрузился в мир децентрализованных технологий и разработки Web3-приложений (dApps). Я успешно освоил современный и востребованный стек технологий, включающий блокчейн Solana, фреймворк Anchor, системный язык программирования Rust, а также продвинутые инструменты frontend-разработки (React, Vite, TypeScript). Я научился проектировать архитектуру децентрализованных систем, писать безопасные смарт-контракты, интегрировать их с клиентскими приложениями и тестировать комплексные программные решения.

Работа над проектом системы электронного голосования позволила мне на практике убедиться в огромном потенциале и высокой социальной значимости блокчейн-технологий. Внедрение надежных, прозрачных и защищенных от фальсификаций информационных систем в такие критически

важные сферы, как избирательные процессы, способно кардинально повысить уровень доверия граждан к результатам голосований, минимизировать риски коррупции и стать важным шагом на пути к построению подлинно демократического и открытого цифрового общества.

В процессе прохождения практики были успешно сформированы и развиты ключевые профессиональные компетенции: способность анализировать предметную область, формулировать технические требования, проектировать и разрабатывать программное обеспечение на базе современных вычислительных платформ, проводить многоуровневое тестирование и сопровождать созданный продукт. Я приобрел ценный опыт самостоятельной работы, научился эффективно искать информацию в официальной документации и решать возникающие технические проблемы.

Считаю, что преддипломная практика прошла максимально эффективно. Все поставленные цели и задачи, включая сбор материалов и практическую реализацию проекта, выполнены в полном объеме. Собранный материал и разработанный программный комплекс полностью готовы для использования в качестве основы моей выпускной квалификационной работы.

ПРИЛОЖЕНИЯ

Приложение 1. Структура проекта и архитектура

В процессе разработки децентрализованного приложения (dApp) для электронного голосования была использована следующая файловая и архитектурная структура, типичная для проектов на базе Anchor.

Проект разделен на две основные директории: `programs` (содержит исходный код смарт-контрактов на Rust) и `frontend` (содержит исходный код клиентского веб-приложения на React). Тесты располагаются в директории `tests`.

Взаимодействие компонентов происходит по следующему сценарию:

1. Пользователь через браузер открывает Frontend-приложение (React).
2. Приложение через библиотеку `@solana/wallet-adapter-react` подключается к расширению кошелька в браузере (Phantom).
3. При инициировании голосования, Frontend формирует транзакцию, вызывает методы смарт-контракта через `@project-serum/anchor` и отправляет запрос на подпись в кошелек.
4. После подписи кошельком, транзакция отправляется на RPC-узел сети Solana (Devnet или Mainnet).
5. Валидаторы сети Solana выполняют транзакцию, изменяя состояние аккаунтов `Proposal` и `Vote` смарт-контракта.

Приложение 2. Исходный код смарт-контракта (lib.rs)

Ниже представлен полный исходный код основного смарт-контракта системы электронного голосования, разработанного с использованием фреймворка Anchor. Контракт реализует логику создания голосований и обработки голосов избирателей.

```
use anchor_lang::prelude::*;

declare_id!("7UfykF9iXWorPS7A3SvgZmJzCTCxpVEqfLyBPw4K51YH");

#[program]
pub mod e_voting {
    use super::*;

    pub fn create(ctx: Context<CreateProposal>, pid: u8, description: String) -> Result<()> {
        require!(
            description.as_bytes().len() <=
                Proposal::DESCRIPTION_MAXIMUM_LENGTH,
            EVotingError::DescriptionTooLong
        );

        let proposal = &mut ctx.accounts.proposal;
        proposal.pid = pid;
    }
}
```

```

        proposal.description = description;
        proposal.yes_votes = 0;
        proposal.no_votes = 0;
        proposal.ongoing = true;
        proposal.owner = *ctx.accounts.user.key;

        proposal.bump = ctx.bumps.proposal;

        Ok(())
    }

    pub fn vote_yes(ctx: Context<AddVote>) -> Result<()> {
        vote(ctx, VoteType::YesVote)
    }

    pub fn vote_no(ctx: Context<AddVote>) -> Result<()> {
        vote(ctx, VoteType::NoVote)
    }
}

fn vote(ctx: Context<AddVote>, vote_type: VoteType) -> Result<()> {
    let proposal = &mut ctx.accounts.proposal;
    let vote = &mut ctx.accounts.vote;

    // Голосование должно быть активным
    require!(
        proposal.ongoing == true,
        EVotingError::VotingSessionIsClosed
    );

    vote.user = *ctx.accounts.user.key;
    vote.proposal = proposal.key();
    vote.bump = ctx.bumps.vote;

    match vote_type {
        VoteType::YesVote => {
            vote.vote = VoteType::YesVote;
            proposal.yes_votes =
proposal.yes_votes.checked_add(1).ok_or(EVotingError::MaxYesVotesReached)?;
        }
        VoteType::NoVote => {
            vote.vote = VoteType::NoVote;
            proposal.no_votes =
proposal.no_votes.checked_add(1).ok_or(EVotingError::MaxNoVotesReached)?;
        }
    }

    Ok(())
}

```

```

}

// -----
// Инструкции

#[derive(Accounts)]
#[instruction(pid: u8)]
pub struct CreateProposal<'info> {
    #[account(
        init,
        payer=user,
        space=8 + Proposal::INIT_SPACE,
        seeds=[
            b"proposal",
            user.key().as_ref(),
            &[pid]
        ],
        bump
    )]
    pub proposal: Account<'info, Proposal>,
    #[account(mut)]
    pub user: Signer<'info>,
    pub system_program: Program<'info, System>,
}

#[derive(Accounts)]
pub struct AddVote<'info> {
    #[account(
        mut,
        seeds = [
            b"proposal".as_ref(),
            proposal.owner.as_ref(),
            &[proposal.pid]
        ],
        bump = proposal.bump
    )]
    pub proposal: Account<'info, Proposal>,
    #[account(
        init,
        payer=user,
        space=8 + Vote::INIT_SPACE,
        seeds = [
            b"vote".as_ref(),
            proposal.key().as_ref(),
            user.key().as_ref()
        ],
        bump
    )]
    pub vote: Account<'info, Vote>,
}

```

```

    #[account(mut)]
    pub user: Signer<'info>,
    pub system_program: Program<'info, System>,
}

// -----
// Состояние и структуры данных

#[account]
pub struct Proposal {
    pub pid: u8,
    pub description: String,
    pub yes_votes: u64,
    pub no_votes: u64,
    pub ongoing: bool,
    pub owner: Pubkey,
    pub bump: u8
}

impl Proposal {
    pub const DESCRIPTION_MAXIMUM_LENGTH: usize = 128;

    pub const INIT_SPACE: usize =
        1 + // pid
        4 + Self::DESCRIPTION_MAXIMUM_LENGTH + // description (string
length + content)
        8 + // yes_votes
        8 + // no_votes
        1 + // ongoing
        32 + // owner pubkey
        1; // bump
}

#[derive(AnchorSerialize, AnchorDeserialize, Clone, PartialEq, Eq)]
pub enum VoteType {
    YesVote,
    NoVote,
}

#[account]
pub struct Vote {
    pub user: Pubkey,
    pub proposal: Pubkey,
    pub vote: VoteType,
    pub bump: u8
}

impl Vote {
    pub const INIT_SPACE: usize =

```



```

    32 + // user pubkey
    32 + // proposal pubkey
    1 + // vote enum (1 byte for variant)
    1; // bump
}

// -----
// Ошибки

#[error_code]
pub enum EVotingError {
    #[msg("Voting session is already closed.")]
    VotingSessionIsClosed,
    #[msg("Description exceeds maximum allowed length.")]
    DescriptionTooLong,
    #[msg("Maximum limit of yes votes reached.")]
    MaxYesVotesReached,
    #[msg("Maximum limit of no votes reached.")]
    MaxNoVotesReached,
}

```

Приложение 3. Описание процесса деплоя и запуска приложения

Для развертывания проекта в локальной среде необходимо выполнить ряд шагов. Предполагается, что на рабочей машине установлены Node.js, Rust, Solana CLI и фреймворк Anchor.

1. **Запуск локального валидатора:** Откройте терминал и выполните команду: `solana-test-validator` Это запустит локальный узел Solana для тестирования транзакций.
2. **Сборка и деплой смарт-контракта:** В новом окне терминала перейдите в корневую директорию проекта и выполните команду: `anchor build` Данная команда скомпилирует Rust-код контракта в BPF (Berkeley Packet Filter) байт-код. Затем разверните программу в локальной сети: `anchor deploy` После успешного деплоя будет выведен Program ID, который необходимо скопировать и обновить в макросе `declare_id!` в файле `lib.rs` и в конфигурационных файлах Frontend-приложения.
3. **Запуск клиентского приложения:** Перейдите в директорию `frontend`: `cd frontend` Установите зависимости: `npm install` Запустите сервер разработки Vite: `npm run dev` Приложение будет доступно в браузере по адресу `http://localhost:5173`. Для полноценного использования потребуется расширение кошелька Phantom в браузере, настроенное на локальную сеть (`localhost:8899`), а также небольшое количество тестовых SOL (можно получить командой `solana airdrop 2` в CLI).